EXPERIMENT 7

Academic Year: 2025-26 **Year and Sem: BE VIII**
Subject: Recommendation Systems Lab **Course Code: CSDOL8022**

Aim: Implementation of feature Engineering and preprocessing of data for recommendation systems

Theory: Preprocessing refers to the steps taken to clean and prepare raw data for use in machine learning models. In the context of recommendation systems, preprocessing helps to ensure that the input data is in a suitable format for further analysis and model training.

Key Preprocessing Steps:

1. Handling Missing Data:

○ Implicit vs Explicit Data: Data can be either implicit (e.g., clicks, views) or explicit (e.g., ratings). Many recommendation systems work with sparse data (i.e., missing entries for most user-item interactions). The preprocessing task often involves filling in missing values, using imputation techniques or ignoring the missing entries based on the system's requirements.

■ **Imputation:** Filling missing values with the user's mean rating or global average rating.

■ **Sparse Matrices:** For collaborative filtering methods, a sparse matrix is often created to represent user-item interactions, leaving missing values empty.

2. Normalization and Scaling:

○ In cases where the data has values with vastly different scales (e.g., ratings on a 1-5 scale, views from 1 to millions), normalization is necessary to ensure that no single feature dominates.

Min-Max Scaling: Scales the data between a specific range (e.g., 0 to 1).

Z-score Normalization: Centers the data by subtracting the mean and dividing by the standard deviation, ensuring the data has zero mean and unit variance.

3. Dealing with Categorical Data: ○ Recommendation systems typically have categorical data (e.g., user gender, item category). These features need to be converted into a numerical format for machine learning algorithms to handle them.

■ One-Hot Encoding: Converts categorical values into binary columns (e.g., user gender with 'Male' and 'Female' categories).

■ Label Encoding: Assigns an integer to each category (e.g., categories of items).

4. Timestamp Handling:

○ For temporal recommendation systems, timestamps associated with interactions (e.g., when a user watched a video or bought an item) may be critical.

■ Time-based Features: Extract features like the day of the week, time of day, or recency of interaction to capture temporal patterns.

5. Outlier Removal:

○ Identifying and removing outliers or erroneous data points is important to ensure the recommendation model does not learn from abnormal user behavior (e.g., a user rating everything a perfect 5).

Feature engineering involves creating new features or transforming raw data into more meaningful representations that improve the model's performance. In the context of recommendation systems, feature engineering helps to capture relevant patterns in the data and enhance the model's ability to predict user preferences accurately.

Key Feature Engineering Techniques:

1. User-Based and Item-Based Features:

○ **User Features:** Information about the user that could improve the recommendation, such as demographic data (e.g., age, gender), historical behavior (e.g., previously purchased items, ratings), or the user's preferences.

■ Example: Number of purchases per user, frequency of interactions, or the user's average rating for items.
○ Item Features: Information about the items being recommended, such as category, price, description, or brand.
■ Example: Item popularity, average rating, or category of the item.

2. Interaction Features:

○ User-Item Interaction Features: These are derived from the interactions between users and items. For example:
■ Recency of Interaction: How recently a user interacted with an item.
■ Frequency of Interaction: How many times a user has interacted with a particular item.
■ Rating Patterns: For explicit data, features like whether a user tends to give high or low ratings can be useful.

3. Aggregated Features:

○ Aggregating user-item interactions can help summarize important patterns. Common aggregated features include:
■ User's Average Rating: The average rating a user gives across all items.
■ Item's Average Rating: The average rating an item receives across all users.
■ Popularity: The total number of interactions or ratings an item has received

Conclusion: By focusing on preprocessing and feature engineering, recommendation systems can be built more efficiently and effectively, leading to more accurate and personalized recommendations for users.